Running a Toy Agent Harness on a Claude Max Subscription

Notes for Robin Langer

28 September 2026

Abstract

This note records how `toy_harness.py` — a hand-written agent loop that normally needs an Anthropic API key — was made to run on a Claude Max subscription instead, with no key and no billing setup. The mechanism is three words long. The interesting part is what it costs you, and what that cost teaches about why the Messages API has structured tool calls at all.

1 The question

The toy harness is about 180 lines: a transcript, a loop, two tools (`bash` and `spawn_agent`), and a permission gate. Its model call is an ordinary SDK call:

```
response = client.messages.create(model=MODEL, max_tokens=16000,
                                  tools=tools, messages=messages)
```

That is the Messages API, which bills against credits from `console.anthropic.com`. The machine had no API key — only a Claude Max subscription and the Claude Code CLI, authenticated through the macOS Keychain. The question was whether the subscription could power the harness.

2 What I got wrong first

My first answer was that Max credentials "won't authenticate" the API. That was incorrect, and worth stating plainly because it changes the decision.

Claude Code authenticates to Anthropic using the Max OAuth token. So a subscription credential demonstrably does produce successful calls against API-shaped endpoints — that is how every Claude Code session works. There is no technical wall.

The real constraint is licensing. The subscription pays for inference *through Anthropic's products* (claude.ai, Claude Code, and its SDK), metered by usage limits inside those products. The API meters inference *per token* for code you write yourself. Extracting the Claude Code token from the Keychain and passing it as `ANTHROPIC_AUTH_TOKEN` would route the second through the first's billing. It would very likely work; it is not what the subscription is for, so I declined to wire it up.

That distinction matters because it points at the solution. If the subscription covers inference *through Claude Code*, then the legitimate move is not to borrow Claude Code's credential — it is to call Claude Code.

3 The mechanism

Claude Code has a headless mode. `claude -p PROMPT` runs one non-interactive turn and prints the result. Crucially, `-tools ""` strips its built-in tool set, which reduces it to a plain prompt-in, text-out endpoint:

```
$ claude -p "Reply with exactly the word PONG and nothing else." --tools ""
PONG
```

No API key. No token handling. This is Claude Code being used as Claude Code, scripted through its documented CLI — squarely inside what the subscription covers. Substituting that for the SDK call gives the harness its inference:

```python
def complete(prompt):
    cmd = ["claude", "-p", prompt, "--tools", ""]
    if MODEL:
        cmd += ["--model", MODEL]
    r = subprocess.run(cmd, capture_output=True, text=True, timeout=300)
    if r.returncode != 0:
        raise RuntimeError(f"claude exited {r.returncode}: {r.stderr.strip()[:300]}")
    return r.stdout.strip()   # stderr dropped: it carries settings warnings
```

Note what is absent from that function: no message array, no roles, no content blocks, no `usage` object. One string in, one string out. Every piece of structure the API supplied, the harness must now impose on its own. The result lives in `toy_harness_max.py`, alongside the original.

4 The cost: inventing a tool protocol

A text endpoint returns no `tool_use` blocks. There is no typed field saying *call bash with this command*. So the protocol has to be described in the prompt and parsed back out of prose:

```
You are a small command-line agent. You work by emitting ONE line at a time,
in one of exactly three forms, and nothing else:

    BASH: <a shell command>
    SPAWN: <a complete, self-contained task for a fresh sub-agent>
    ANSWER: <your final answer, one or two sentences>
```

and read back with a regular expression:

```
m = re.match(r"\s*(BASH|SPAWN|ANSWER)\s*:\s*(.*)", line, re.S)
```

This is the ReAct pattern — how essentially every agent worked before tool use was trained into the models. It works, and it is instructive, but it reintroduces a failure mode the API had eliminated. If the model replies *"Let me check: BASH: ls"* or drops the prefix entirely, the regex misses, no command runs, no error is raised, and the malformed text sits in the transcript skewing every later turn.

That failure is not hypothetical even on the current API. On Claude Opus 5 with thinking disabled, the model occasionally writes a tool call into its *visible text* rather than emitting a `tool_use` block: the turn completes normally, the call never executes, and nothing reports

a problem. Structured tool use is precisely the fix for that class of bug — which is why it resurfaces the moment you take the structure away. The prose-protocol harness has it by construction, on every single turn.

5 Side by side

	`toy_harness.py`	`toy_harness_max.py`
Credential	API key + credits	Max subscription
Inference	`client.messages.create`	`claude -p -tools ""`
Transcript	`list[dict]`, typed	one `str`, rebuilt per turn
Model's intent	`block.type == "tool_use"`	regex over one line
Malformed output	impossible (API validates)	silently does nothing
Parallel calls	native, many blocks per turn	one line per turn
Token accounting	`response.usage`	none exposed

The loop, the gate, the recursion, and the five underlying concepts are identical in both files. Only the credential and the encoding of intent differ.

6 Verification

Both files were run on the machine. A direct query:

```
$ python3 toy_harness_max.py "How many Python files are in this directory,
                              and how many lines is the biggest one?"
+- agent(depth=0): How many Python files are in this directory, and how many
|  BASH: find . -name '*.py' -type f | wc -l && find . -name '*.py' -type f ...
|  <- '2\n     161 ./toy_harness_max.py\n     207 ./toy_harness.py\n 368 total'
|  ANSWER: There are 2 Python files, and the biggest is ./toy_harness.py at 207 lines.
+- done (2 turns)
```

And a delegated one, which exercises the sub-agent path:

```
$ python3 toy_harness_max.py "Delegate to a sub-agent: have it report the total
                              line count of every .py file here. Then state it."
+- agent(depth=0): Delegate to a sub-agent: have it report the total line count
|  SPAWN: In the directory /Users/robin/git/harness, find every file with a .py
   +- agent(depth=1): In the directory /Users/robin/git/harness, find every .py
   |  BASH: find /Users/robin/git/harness -name '*.py' -type f | wc -l; ...
   |  <- '2\n     368'
   |  ANSWER: 2 Python files, 368 total lines.
   +- done (2 turns)
|  <- '2 Python files, 368 total lines.'
|  ANSWER: 368 lines total across the 2 .py files.
+- done (2 turns)
```

Two details in that second trace are worth dwelling on.

First, the sub-agent ran a command and reasoned about the output, but the parent's transcript gained exactly one line: the returned string. That is the whole economic argument for delegation — the child's work happened, and the parent paid roughly fifteen tokens for it. A sub-agent is not a special mechanism; it is the same function called with a fresh transcript, so its turns cannot enter the parent's.

Second, look at what the parent had to write in its `SPAWN:` line: the absolute path `/Users/robin/git/harness`. It could not say "here". The sub-agent's isolation is not a policy decision that could be relaxed — the child's prompt was built from scratch, so nothing about the parent's situation exists inside it unless it was spelled out.

7 Practical notes

Latency. Each `complete()` call starts a whole Claude Code process, so turns take seconds rather than milliseconds. Fine for a teaching harness; not a serving architecture.

Model selection. `TOY_MODEL=claude-haiku-4-5` switches backends in both files. The loop is byte-identical whichever model you point it at, so experiment cheaply and switch up when you want better reasoning.

A real bug found along the way. Every `claude -p` invocation printed this to stderr:

```
Permission allow rule (~/.claude/settings.json):
  Write(/Users/robin/git/**) is not matched by file permission
  checks -- only Edit(path) rules are. Use Edit(/Users/robin/git/**)
  instead (Edit rules cover all file-editing tools).
```

That is a genuine defect in the global settings, not in the harness. All four `Write(...)` allow rules are inert, because file-permission checks match only `Edit(path)` rules — and `Edit` already covers every file-editing tool, `Write` included. The practical effect is permission prompts for writes that were meant to be pre-approved.

8 Where to take it next

Adding *compaction* is the sharpest follow-on, and it is easier in the Max version than the API one: the transcript there is a literal string, so you can watch it cross a size threshold, get summarised by a nested `complete()` call, and be replaced in place. Roughly twenty lines, and it makes the central tension of the whole design concrete — everything must be in the transcript to exist at all, and the transcript must stay small to remain useful.